Algorithmique

Chapitre A2 : Intelligence artificielle

Eric Brillaux
Lycée Théophile Gautier PC
2025 - 2026
Version : 27 mars 2026

Prérequis de PCSI

▶

Programme officiel

Notions	Commentaires
Algorithme des k plus proches voisins avec distance euclidienne.	Matrice de confusion. Lien avec l'apprentissage supervisé.
Algorithme des k-moyennes.	Lien avec l'apprentissage non-supervisé. La démonstration de la convergence n'est pas au programme. On observe des convergences vers des minima locaux.
Mise en œuvre	
La connaissance dans le détail des algorithmes de cette section n'est pas un attendu du programme. Les étudiants acquièrent une familiarité avec les idées sous-jacentes qu'ils peuvent réinvestir dans des situations où les modélisations et les recommandations d'implémentation sont guidées, notamment dans leurs aspects arborescents.	

Sommaire

L'expression *intelligence artificielle* (IA) a été créée par les américains John McCarty et Marvin Lee Minsky en 1956. Cette époque est marquée par un forte enthousiasme dans le domaine. Les premières IA, comme Eliza, construite au MIT en 1965, étaient capable de dialoguer en anglais en reconstruisant des phrases toutes faites. Pourtant, les intelligences artificielles vraiment performantes ont mis du temps à émerger. À partir des années 80, des outils formels informatiques sont développer pour étudier l'IA : les réseaux de neurone, les algorithmes génératifs, etc. L'IA a réussi à supplanter l'humain dans certains jeux réputés réservés à l'intelligence humaine : citons la machine Deep Blue qui battit G. Kasparov aux échecs en 1997, ou encore la machine AlphoGo qui battit L. Sedol au jeu de go en 2016.

I Introduction **1**
I.A Apprentissage automatique 2
I.B Algorithmes de classement 3
II Apprentissage supervisé **5**
II.A Algorithme des k plus proches voisins 5
II.B Matrice de confusion 9
III Apprentissage non supervisé **12**
III.A Algorithme des k moyennes 13

I – Introduction

Définition A2.1

*L'**intelligence artificielle** désigne une machine capable de réaliser des tâches habituellement exécutées par des humains, qui nécessitent des capacités de raisonnement, de mémoire, et d'apprentissage,*

En particulier, on souhaite rendre une machine capable d'accéder à certaines connaissances sans avoir été programmée explicitement pour cela, autrement dit capable d'apprendre par elle-même. Les exemples d'applications de l'IA sont légion. Parmi les grands domaines de l'IA, citons

- la reconnaissance de formes, i.e. de texte, de son (reconnaissance vocale, Alexa, Siri) ou d'images (reconnaissance de visages par la police, analyse d'images médicales) ;
- le traitement du langage naturel, i.e. la reconnaissance vocale, la synthèse vocale (personnes ne pouvant plus parler, génération de chanson), la traduction, les agents conversationnels (chatGBT, chatbots de nombreux sites et applications) ;
- les algorithmes de recherche, i.e. google, les filtres de spam ;
- la génération de contenu (chatGPT, images artistiques, chansons).

Comme application pluridisciplinaire, citons les voitures autonomes et les robots de compagnie ou d'assistance, qui utilisent le traitement du langage naturel (conversation) et la reconnaissance de formes (mouvement, détection d'obstacle).

(a) Voitures autonomes

(b) Reconnaissance vocale

(c) Agent conversationnel

(d) Génération d'images artistiques

Fig. A2.1 – Quelques domaines d'applications de l'intelligence artificielle (IA). (a) Voitures autonomes. (b) Reconnaissance vocale. (c) Agent conversationnel. (d) Génération d'images artistiques.

I.A – Apprentissage automatique

L'apprentissage automatique (*machine learning*) est un domaine majeur de l'intelligence artificielle.

Définition A2.2

*L'**apprentissage automatique** désigne une méthode qui permet à une IA d'apprendre à résoudre un problème sans jamais la programmer explicitement pour résoudre ce problème particulier.*

Une machine **apprend** lorsqu'elle améliore son fonctionnement dans le cadre de la résolution d'un problème : des outils statistiques sont appliqués sur des données et la machine modifie son fonctionnement à partir des résultats de traitement pour améliorer son efficacité. Pour trouver les bons outils statistiques, un modèle est construit à partir des données du problème.

- L'**algorithme traditionnel** prend en entrée des données et se sert de règles (condition `if`, boucle `for`, fonctions mathématiques, etc.) pour produire une réponse en sortie.
- L'**algorithme d'IA**, lors de la phase d'apprentissage, prend en entrée des données (en grande quantité) et éventuellement des réponses (si les données sont déjà étiquetées) et en déduit des règles (de régression, de classification, etc.).

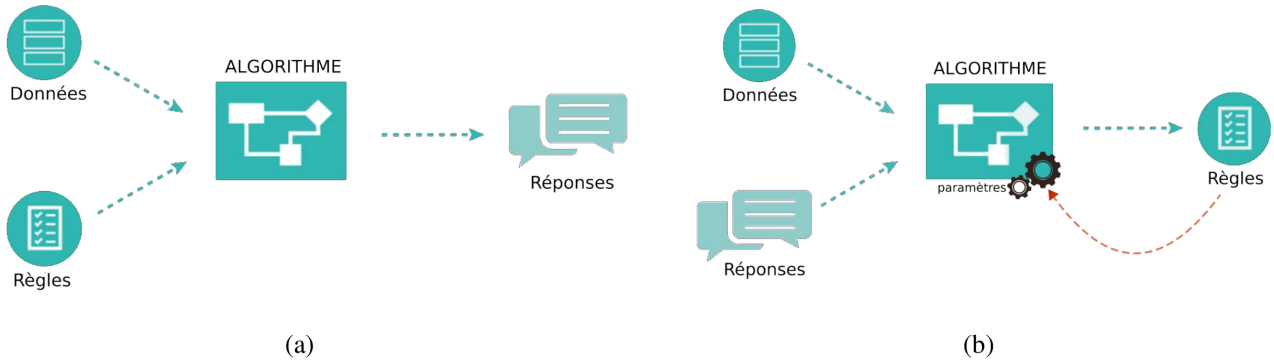


Fig. A2.2 – (a) Principe d'un algorithme traditionnel. L'algorithme prend en entrée des données et se sert de règles (condition `if`, boucles `for`, fonctions mathématiques, etc.) pour produire une réponse en sortie. (b) Principe de l'apprentissage d'un algorithme d'intelligence artificielle. Lors de la phase d'apprentissage, l'algorithme prend en entrée des données (en grande quantité) et éventuellement des réponses (si les données sont déjà étiquetées) et en déduit des règles (de régression, de classification, etc.). Lors de la phase d'apprentissage, l'algorithme modifie les règles en affiner la valeur des paramètres de l'algorithme.

On caractérise un algorithme d'apprentissage automatique selon sa fonction et la méthode d'apprentissage. Il existe trois techniques principales d'apprentissage automatique. La **fonction** d'un algorithme détermine la nature des règles et l'objectif de l'algorithme. On distingue les algorithmes de

- **régression** : prédire une nouvelle donnée y à partir d'une donnée x issue d'une variable réelle continue.
- **classification** : répartir une nouvelle donnée dans une classe.

Dans la suite, nous nous intéresserons aux algorithmes de classification. Nous définirons la notion de classification dans la Sec. I.B. La **méthode d'apprentissage** détermine la façon dont l'algorithme apprend. On distingue les algorithmes

- apprentissage **supervisé** : l'algorithme apprend à partir d'un ensemble de données étiquetées (on donne les réponses à l'algorithme). Voir Sec. **??**.

- apprentissage **non supervisé** : l'algorithme apprend à partir d'un ensemble de données non étiquetées (l'algorithme doit trouver des règles qui fournissent des réponses pertinentes). Voir Sec. **??**.

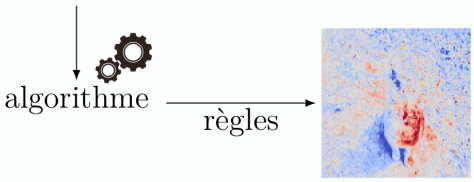
Remarque

Il existe d'autres types d'apprentissage comme l'apprentissage par renforcement et l'apprentissage en profondeur.

Exemple A2.1 : Reconnaissance d'images

Un application importante de l'IA est la reconnaissance d'image. On montre ci-dessous une illustration de la phase d'apprentissage automatique d'un algorithme de reconnaissance d'image.

(a) cat (b) cat (c) cat (d) dog (e) dog (f) dog



Une fois l'algorithme entraîné, on exige de l'algorithme qu'il puisse fournir une réponse en sortie lorsqu'on lui donne une image inconnue en entrée, comme ci-dessous.

1) S'agit-il d'un problème de régression ou de classification ?
2) S'agit-il d'un apprentissage supervisé ou non supervisé ?

Solution

Algorithme de classification par apprentissage supervisé.

I.B – Algorithmes de classement

Dans le cadre du cours, nous étudierons des problèmes de classement, plus spécifiquement des problèmes de classement exclusif, ou chaque donnée ne peut exister que dans un seul groupe. Considérons un ensemble de données E (images, sons, textes...) de N éléments.

Définition A2.3 (Classification)

*On appelle **classification** des données une partition de E, c'est-à-dire une famille de sous-ensembles $E_i \subset E, i = 1, ..., n$:*

- *dont la réunion décrit l'ensemble entier (tous les éléments sont classés) :* $\cup_i E_i = E$ *;*
- *disjoints (un élément n'appartient qu'à une seule classe) :* $E_i \cap E_j = \varnothing$ *si* $i \neq j$.

On peut représenter un ensemble d'apprentissage E de N données en `python` par une liste `[y1,y2,...,yN]`. Un méthode efficace pour définir une partition sous `python` est d'utiliser un dictionnaire.

Propriété A2.1

On représente une classification $(E_i)_{i=1,\dots n}$ *sous* `python` *par un* ***dictionnaire***

```
p={y1:c1, y2:c2, ..., yN:cN}
```

où les y_j *sont les éléments de* $E = \cup_i E_i$ *et* c_j *désigne la classe à laquelle appartient* y_j *(l'un des* E_i*).*

Remarque

La recherche de la classe d'un élément x *de* E *exige alors une complexité constante* $O(1)$*, grâce à la structure de dictionnaire. En définissant la partition comme une liste* `p=[[y1,c1], [y2,c2], ..., [yN,cN]]`*, on obtiendrait une complexité linéaire en* $O(N)$.

Exemple A2.2 : Reconnaissance d'images

Quelle est la classification des images de l'exemple précédent ? La définir sous `python`.

Solution

La classification est $E = \text{chats} \cup \text{chiens}$ *où* $\text{chats} = \{a, b, c\}$ *contient les 3 premières images et* $\text{chiens} = \{d, e, f\}$ *les 3 suivantes. Sous python, on peut définir l'ensemble de données* E *sous forme d'une liste de caractères désignant chaque image,*

```
E = ['a', 'b', 'c', 'd', 'e', 'f'].
```

Les classes sont les chaînes de caractères `'chat'` *et* `'chien'`*. La classification est définie par le dictionnaire*

```
p = {'a':'chat', 'b':'chat', 'c':'chat', 'd':'chien', 'e':'chien', 'f':'chien'}.
```

Exemple A2.3 : Recherche d'articles de presse

On souhaite proposer un algorithme de recherche d'articles de presse suivant leur sujet. On considère 5 particles de presse appartenant à 2 classes : les sujets « culture » et « science ». Les articles sont modélisés par un point $y = (c, s)$ du plan où c représente la fréquence du mot « culture » et s la fréquence du mot « science » dans le corps de l'article.

Articles	Article 1	Article 2	Article 3	Article 4	Article 5
c	0.8	0.2	0.6	0.4	0.9
s	0.5	0.9	0.3	0.7	0.2
Sujet	Culture	Science	Culture	Science	Culture

1) Définir formellement puis en `python` l'ensemble d'apprentissage E.
2) Définir formellement puis en `python` la classification $(E_i)_{i=1,\dots n}$ associée.

Solution

1) *On peut coder l'ensemble E sous python avec une liste,*

```
E = [(0.8,0.5), (0.2,0.9), (0.6,0.3), (0.4,0.7), (0.9,0.2)].
```

2) *On peut ensuite définir la classification mathématiquement par $E = \text{culture} \cup \text{science}$ où $\text{culture} = \{(0.8, 0.5), (0.6, 0.3), (0.9, 0.2)$ et $\text{science} = \{(0.2, 0.9), (0.4, 0.7)\}$. On peut définir la classification sous python par le dictionnaire*

```
p = {(0.8,0.5):'culture', (0.2,0.9):'science', (0.6,0.3):'culture', (0.4,0.7):'science',
                                    (0.9,0.2):'culture'} .
```

II – Apprentissage supervisé

Nous nous intéressons ici à l'apprentissage supervisé des algorithmes de classement. Le développement d'un algorithme d'IA comprend deux phases :

- l'apprentissage (Sec. II.A) ;
- le test (Sec. II.B) ;

L'algorithme de classement standard pour réaliser un apprentissage supervisé est l'algorithme des k plus proches voisins.

II.A – Algorithme des k plus proches voisins

(i) Notion de distance — Considérons un ensemble de données E contenant N éléments, et une classification de E, noté $(E_i)_{i=1,\dots n}$. Les donnés sont donc réparties en n classes bien déterminées. Cet ensemble E est appelé **ensemble d'apprentissage**.

Soit une donnée x qui n'appartient pas à E. On souhaite classer x selon la classification $(E_i)_{i=1,\dots n}$ c'est-à-dire trouver la classe i dont les éléments sont les « plus proches » de x. Pour cela, il faut définir une notion de distance, qui dépend de la nature des données de E et du problème.

Définition A2.4 (Distance)

*Une **distance** est une fonction $d(x, y)$ de deux données x et y de l'ensemble d'apprentissage E.*

Lorsque les données sont des nombres entiers ou réels, il est naturel d'utiliser la distance **euclidienne**. Pour des points $x = (x_1, x_2)$ et $y = (y_1, y_2)$ du plan, par exemple, la distance euclidienne est définie par

$$\boxed{d(x, y) = \sqrt{(x_1 - y_1)^2 + (x_2 - y_2)^2}.} \tag{1}$$

Exemple A2.4 : Exemples de distance

1) Pour des points du plan $x = (x_1, x_2)$ et $y = (y_1, y_2)$ définis sur une grille, on peut utiliser la distance de Manhattan, définie par

$$d_{\mathrm{M}}(x, y) = |x_1 - y_1| + |x_2 - y_2|. \tag{2}$$

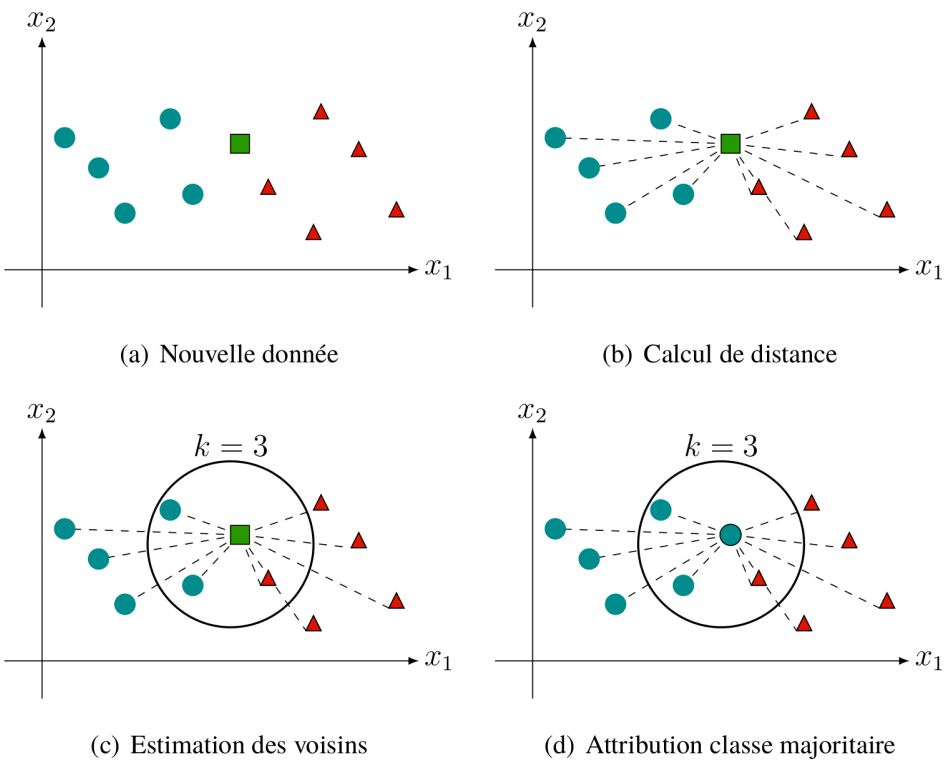
Justifier le nom donner à cette distance. Que vaut $d_{\mathrm{M}}((1, 2), (3, 4))$?

2) On suppose que les données de E sont des images en niveau de gris. En supposant les couleurs codées sur 8 bits, dans quel ensemble mathématique appartient le niveau de gris $G(x)$ d'un pixel x ? Définir une distance d_{g} qui compare les niveaux de gris des mêmes pixels x et y pris de deux images différentes.
3) On suppose que les données de E sont des chaînes de caractères. On définit la distance de Hamming $d_{\mathrm{H}}(x, y)$ comme le nombre de caractères à modifier pour passer de la chaîne x à la chaîne y. Par exemple, la distance de Hamming entre les mots « ramer » et « cases » vaut 3. Que vaut la distance de Hamming pour : les mots « chanter » et « chat » ; les nombres binaires 0110 et 1110 ?

Solution

1) $G(x) \in [0, 255]$. *La différence de niveau de gris entre deux pixels x et y peut s'évaluer par la distance* $d_{\mathrm{gris}}(x, y) = |G(x) - G(y)|$.

(ii) Principe de l'algorithme — L'algorithme des k plus proches voisins est paramétré par un entier $k < N$. On suppose avoir défini une distance d sur l'ensemble d'apprentissage. Soit un élément x n'appartenant pas à E à classer (carré vert sur la Fig. A2.4(a)). On commence par calculer les distances $d(x, y)$ entre x et chaque élément y de E (Fig. A2.4(b)). On détermine ensuite les k plus proches voisins (Fig. A2.4(c)). Puis on détermine la classe majoritaire des voisins (Fig. A2.4(d)).



(a) Nouvelle donnée (b) Calcul de distance

(c) Estimation des voisins (d) Attribution classe majoritaire

FIG. A2.3 – Principe de l'algorithme des k plus proches voisins. (a) On considère un ensemble d'apprentissage E contenant N éléments. (b) On souhaite classer un nouvel élément x n'appartenant pas à E. (c) On détermine les k plus proches voisins de x dans E. On associe à x la classe majoritaire des voisins. (d) On associe à x la classe majoritaire des voisins.

Définition A2.5 (Algorithme des k plus proches voisins)

1. *On trie l'ensemble E suivant l'ordre croissant des distances à x, c'est-à-dire la valeur de $d(x, y)$ avec $y \in E$. On obtient une liste à N éléments* $\text{tri}(E, x, d)$.
2. *On choisit les k plus proches voisins, c'est-à-dire les k premiers éléments de* $\text{tri}(E, x, d)$. *On obtient la liste* $\text{voisins}(E, x, d, k)$.
3. *On détermine la classe dominante de* $\text{voisins}(E, x, d, k)$, *notée* $\text{classemaj}(E, x, d, k, p)$ *c'est-à-dire qu'on détermine la classe i telle que l'intersection $E_j \cap \text{voisins}(E, x, d, k)$ contienne le maximum d'éléments pour $j = i$. On associe ainsi la classe i au nouvel élément x.*

Nous implémenterons en `python` les fonctions $\text{tri}(E, x, d)$, $\text{voisins}(E, x, d, k)$ et $\text{classemaj}(E, x, d, k, p)$ en TD. Pour l'instant, nous allons appliquer l'algorithme des k plus proches voisins sur un exemple simple sans se soucier de l'implémentation des fonctions.

Exemple A2.5 : Recherche d'articles de presse

Reprenons l'exemple de recherche d'articles de presse, Ex. I.B. Un auteur publie un nouvel article sur le site internet du magazine (Article 6) dont la fréquence du mot « culture » est $c = 0.6$ et la fréquence du mot « science » est $s = 0.5$. On souhaite classer cette nouvelle donnée $x = (0.6, 0.5)$ dans l'une des deux classes « culture » ou « science » en s'aidant de l'ensemble d'apprentissage E précédent. On utilise la distance euclidienne.

1) Rappeler la définition de la distance euclidienne dans ce contexte.
2) Déterminer la liste triée des éléments de E suivant l'ordre croissant des distances à x. Représenter l'ensemble des données E et x sur un graphique.
3) Déterminer les $k = 3$ plus proches voisins de x dans E. Représenter les k plus proches voisins sur le graphique. Pourquoi est-il préférable de choisir un nombre impair k de voisins ?
4) Déterminer la classe majoritaire des voisins. En déduire la classe de x.

Solution

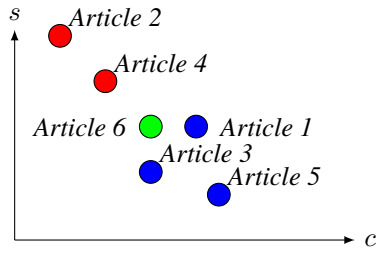
1) La distance euclidienne entre deux points $x = (c_1, s_1)$ et $y = (c_2, s_2)$ du plan est donnée par

$$d(x, y) = \sqrt{(c_1 - c_2)^2 + (s_1 - s_2)^2}.$$

2) On calcule les distances entre x et chaque élément de E. On obtient la liste triée suivante

$$\boxed{\text{tri}(E, x, d) = [(0.4, 0.7), (0.7, 0.5), (0.6, 0.3), (0.8, 0.5), (0.2, 0.9)]\,.}$$

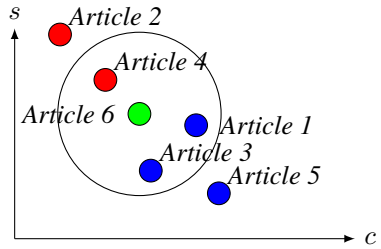
On représente les données sur le graphique ci-dessous.



3) Les $k = 3$ plus proches voisins de x sont les trois premiers éléments de la liste triée. On obtient

$$\boxed{\text{voisins}(E, x, d, 3) = [(0.7, 0.5), (0.6, 0.3), (0.8, 0.5)]\,.}$$

On représente les k plus proches voisins sur le graphique ci-dessous.



Il est préférable de choisir un nombre impair de voisins pour éviter les situations d'égalité.

4) La classe majoritaire des voisins est la classe « culture ». On en déduit que l'article 6 est un article de culture.

$$\boxed{\text{classe}(E, x, d, 3) = \textit{culture}.}$$

(iii) Critique de l'algorithme — Cet algorithme possède certains avantages et inconvénients.

▶ Avantages.

1 L'algorithme est *simple*, et facile à implémenter, comme nous le verrons en TD.

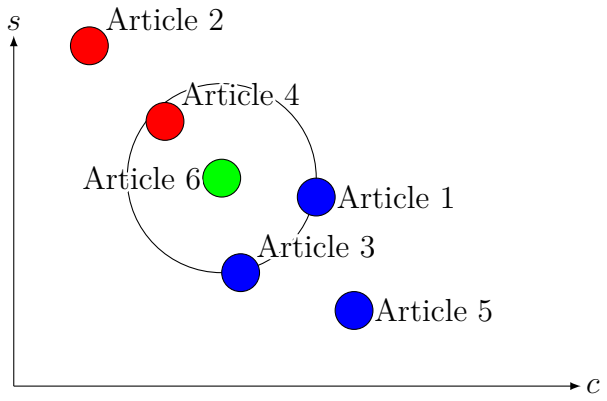
2 L'algorithme nécessite *peu de paramètres* : la distance d et le nombre de voisins k.

▶ Inconvénients.

1 L'algorithme est *paresseux* : au lieu de subir une longue phase d'entraînement pour optimiser les paramètres de l'algorithme, l'algorithme sollicite l'ensemble d'apprentissage entier pour réaliser une prédiction. Solliciter une quantité aussi importante de données à chaque fois peut être coûteux en mémoire, en temps et en argent.

2 L'algorithme fonctionne mal avec des données de *grande dimension* (grand nombre de coordonnées).

3 L'algorithme est *sensible* au choix du nombre de voisins k. Si l'on avait choisi $k = 1$, on aurait trouvé dans l'exemple précédent que le plus proche voisin du nouvel article est $(0.7, 0.5)$, qui est un article de science. L'algorithme aurait donc classé l'article 6 comme un article de science.



4 Le choix de la *classe majoritaire* n'est pas forcément le meilleur critère. En effet, ce choix ne prend pas en compte la distribution des classes des voisins. Comme le montre l'exemple ci-dessus, la classe majoritaire ne correspond pas forcément à la classe du plus proche voisin. Pour améliorer la détermination de la classe, on peut par exemple pondérer les classes par la distance entre les voisins et le point x à classer.

II.B – Matrice de confusion

(i) Définitions et exemples — L'algorithme des k plus proches voisins permet d'effectuer des prédictions pendant la phase d'apprentissage finie. Afin d'améliorer ou de contrôler le bon fonctionnement de l'IA, il faut mesurer la qualité de ces prédictions. Pour cela, on introduit la notion de matrice de confusion.

Exemple A2.6 : Test médical

Considérons un test médical destiné à déterminer la présence ou non d'une infection chez un groupe de 100 patients. L'efficacité du test n'est pas parfaite. On suppose que 90 prédictions du test sont corrects, tandis que 10 prédictions sont fausses.

- Parmi les tests corrects, 20 sont de vrai positifs, et 70 des vrais négatifs.
- Parmi les tests incorrects, 8 sont de faux négatifs, et 2 des faux positifs.

Représenter les données dans le tableau suivant.

		Classe prédite	
		Infecté	Sain
Classe réelle	Infecté	...	...
	Sain	...	...

Solution

		Classe prédite	
		Infecté	*Sain*
Classe réelle	*Infecté*	20 *(vrai positif)*	8 *(faux négatif)*
	Sain	2 *(faux positif)*	70 *(vrai négatif)*

On peut généraliser cet exemple à des données issues de prédictions d'une IA. Considérons un ensemble données, appelé ensemble de test T. Notons $(T_i)_{i=1,\dots n}$ la classification de T. Les données test sont donc réparties en n classes bien déterminées. On applique un algorithme d'apprentissage, qui à chaque donnée $x \in T$ associe une classe T_j, pour un certain $j = 1, \dots n$. On appelle T_i la classe réelle de x et T_j la classe prédite de x.

Définition A2.6 (Matrice de confusion)

*On appelle **matrice de confusion** la matrice M de taille $n \times n$ (avec n le nombre de classes) dont*

- *chaque ligne i correspond à une classe réelle T_i ;*
- *chaque colonne j correspond à une classe prédite T_j ;*
- *chaque élément M_{ij} donne le nombre de données de classe réelle T_i et de classe prédite T_j.*

Dans une matrice de confusion, chaque élément de la **diagonale** correspond à des points correctement classés. Chaque élément **hors diagonal** correspond à des points mal classés. À partir de la matrice de confusion, on peut définir plusieurs grandeurs pour quantifier la qualité des prédictions.

- Le **taux d'erreur** TE est le nombre de prédictions incorrectes N_i sur le nombre total de prédictions N,

$$\boxed{\text{TE} = \frac{N_i}{N}.}$$

  On vise un taux d'erreur le plus proche possible de 0.
- La **précision** est le nombre de prédictions correctes N_c sur le nombre total de prédictions N,

$$\boxed{\text{P} = \frac{N_c}{N} = 1 - \text{TE}.}$$

Remarque

Dans le cas où il n'existe que deux classes, « positif » et « négatif », on définit aussi les grandeurs suivantes.

- *La **sensibilité** est le nombre de positifs corrects p_c sur le nombre total de positifs p,*

$$s = \frac{p_c}{p}. \tag{3}$$

  Le taux de faux positifs vaut alors TFP $= 1 - s$.
- *La **spécificité** est le nombre de négatifs corrects n_c sur le nombre total de négatifs n,*

$$\overline{s} = \frac{n_c}{n}. \tag{4}$$

Exemple A2.7 : Caractéristiques d'un test médical

Considérons le test médical suivant.

		Classe prédite	
		Infecté	Sain
Classe réelle	Infecté	20 (vrai positif)	8 (faux négatif)
	Sain	2 (faux positif)	70 (vrai négatif)

1) Que vaut le nombre de données N ? Que vaut le nombre de données correctes N_c ? Que vaut le nombre de données incorrectes N_i ?
2) Calculer la précision P et le taux d'erreur TE.
3) Calculer le nombre de négatifs n et le nombre de positifs p. En déduire la sensibilité s et la spécificité $\overline{s}$.
4) On définit le taux de négatifs $\mathrm{TN} = \frac{n}{N}$. Trouver une relation littérale entre s, P et TN.

Solution

1) Le nombre de données test vaut $N = 100$, pour $N_i = 10$ données incorrectes et $N_c = 90$ données correctes.

2) Le taux d'erreur vaut donc $TE = 10/100 = 0{,}1$, pour une précision $P = 90/100 = 0{,}9$.

3) Le nombre de prédictions positives correctes vaut $p_c = 20$, tandis que le nombre total de prédictions positives vaut $p = 22$. La sensibilité vaut donc $s = 20/20 = 0{,}91$. Le nombre de prédictions négatives correctes vaut $\overline{p}_c = 70$, tandis que le nombre total de prédictions négatives vaut $\overline{p} = 78$. La spécificité vaut donc $\overline{s} = 70/78 = 0{,}90$.

4) On a également les relations $p+n = N$ et $p_c+n_c = N_c$. Ainsi, $\overline{s} = -\frac{p_c}{n} + \frac{N_c}{n} = -s\frac{p}{n} + P\frac{N}{n} = -s(\frac{N}{n}-1) + P\frac{N}{n} = (P-s)\frac{N}{n} + s$ où $\frac{n}{N} = TN$ représente le taux de négatifs. Ainsi,

$$\boxed{\overline{s} = s + \frac{P-s}{TN}.}$$

(ii) Implémentation — La détermination de la matrice de confusion peut s'implémenter de la façon suivante. On peut représenter la matrice de confusion par une liste de liste. La fonction prend en argument : l'ensemble d'apprentissage E, l'ensemble test T, la distance d, le nombre de plus proches voisins k, et la partition (classification) p. Pour simplifier, les classes sont associées à des entiers $i, j = 0, ..., n-1$.

```
def confusion(E, T, d, k, p, n):
    """
    Entrées : ensemble d'apprentissage E, ensemble de test T, distance d,
              nombre de voisins k, dictionnaire de classes p, nombre de classes n
    Sortie : matrice de confusion M
    """
    M = [[0]*n for l in range(n)]    # initialisation matrice
    for x in T:                      # boucle sur ensemble test
        i = p[x]                     # classe réelle
        j = classemaj(E,x,d,k,p)     # classe prédite
        M[i][j] += 1                 # ajout au compte
    return M                         # retourne matrice de confusion
```

Connaissant la matrice de confusion, on peut ensuite calculer le taux d'erreur ou la précision, comme dans l'exemple ci-dessous. La fonction `precision` prend en argument la matrice de confusion M et le nombre de prédictions N, et renvoie la précision P.

```python
def precision(M,N):
    """
    Entrée : M = matrice de confusion, N = nombre de prédictions
    Sortie : précision = Nc/N
    """
    n    = len(M)                        # nombre de classes
    diag = [M[i][i] for i in range(n)]   # diagonale = prédictions correctes
    Nc   = sum(diag)                     # nombre prédictions correctes
    return Nc/N                          # précision
```

Exemple A2.8 : Implémentation de test de qualité des prédictions

Pour chacune des fonctions ce-dessous, on précisera les spécifications de cette fonction et on la commentera. Tester cette fonction sur un exemple.

1) Définir en `python` une fonction `erreur` qui prend comme seul argument la matrice M *de taille quelconque* et qui calcule le taux d'erreurs du test.
2) Définir en `python` une fonction `sensibilite` qui prend comme seul argument la matrice M de taille 2 par 2 et qui calcule la sensibilité du test.
3) Définir en `python` une fonction `specificite` qui prend comme seul argument la matrice M de taille 2 par 2 et qui calcule la spécificité du test.
4) Définir en `python` une fonction `prevalence` qui prend comme seul argument la matrice M de taille 2 par 2 et qui calcule la prévalence du test, c'est-à-dire le nombre de patients infectées sur le nombre total de patients.

Solution

```python
def erreur(M):
    n = len(M)
    N = sum(sum(M))
    TE = 0
    for i in range(N):
        for j in range(N):
            if i != j:
                TE += M[i][j]
    return TE/N
def sensibilite(M):
    return M[0][0]/(M[0][0]+M[1][0])
def specificite(M):
    return M[1][1]/(M[0][1]+M[1][1])
def prevalence(M):
    N = M[0][0] + M[0][1] + M[1][0] + M[1][1]
    return (M[0][0]+M[0][1])/N
```

III – Apprentissage non supervisé

L'apprentissage **non supervisé** utilise des algorithmes d'apprentissage automatique pour analyser et regrouper des jeux de données non étiquetés. Nous appliquerons ces algorithmes dans des problèmes de

classement, plus spécifiquement au classement exclusif, ou chaque donnée ne peut exister que dans un seul groupe. Dans cette partie, nous nous intéresserons uniquement à la phase d'apprentissage de l'algorithme (la phase de test est similaire à celle de l'apprentissage supervisé avec la matrice de confusion).

III.A – Algorithme des k moyennes

(i) Principe de l'algorithme — Dans un apprentissage non supervisé, c'est à la machine de trouver des éléments de similarités dans les données et d'en déduire les classes. La partition p n'est donc pas donnée ! On impose par contre à la machine le nombre k de classes à définir.

Pour partitionner les données, on initialise chaque groupe i par une donnée centrale c_i pour $i = 1, ..., k$. Ces centres peuvent par exemple être choisis aléatoirement dans l'ensemble d'apprentissage E. On définit ensuite la partition initiale en classant les points de E selon leur proximité avec les centres c_i. La proximité entre les données se mesure, comme pour l'apprentissage supervisé, avec une distance d. À chaque itération, on affine le partition en calculant pour chaque groupe les barycentres des groupes, qui servent de nouveaux centres pour le partitionnement.

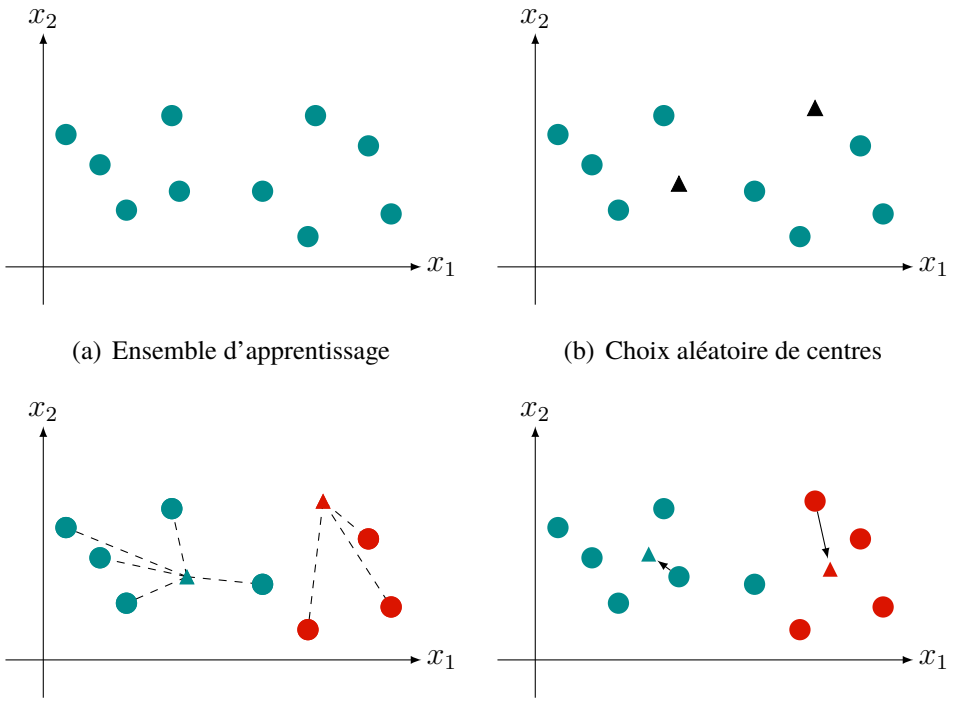
Définition A2.7 (Algorithme des k moyennes)

Soit un ensemble d'apprentissage E, deux entiers $k \geq 1$ et $n_i \geq 1$, et d une distance.

1. *Initialiser aléatoirement les centres c_i parmi les éléments de E pour $i = 1, ..., k$.*
2. *À chaque itération$j = 1, ..., n_i$, effectuer :*
   - *partitionnement de E autour des centres c_i selon la distance d ;*
   - *ré-évaluation des centres c_i comme barycentre du groupe E_i.*

Remarque

Pour pouvoir définir un barycentre, il faut que E provienne d'un ensemble dont la structure est celle d'un espace affine. En particulier, il doit exister une opération $+$ dans E, ce qui permet d'additionner des points, et une opération de multiplication par un scalaire.



(a) Ensemble d'apprentissage (b) Choix aléatoire de centres

(c) Partitionnement suivant la distance (d) Évaluation des centres comme barycentres

FIG. A2.4 — Principe de l'algorithme des k moyennes. (a) On considère un ensemble d'apprentissage E contenant N éléments. (b) On choisit aléatoirement k centres parmi les données de E. (c) On partitionne E suivant la distance aux centres. (d) On ré-évalue les centres comme barycentre de chaque classe. On réitère ensuite le processus.

(ii) Programmation — On peut programmer chaque étape de la façon suivante.

1 Commençons par initialiser les centres à partir d'un ensemble d'apprentissage E et d'un nombre de classes k. Pour cela, on peut utiliser la fonction `sample` de la bibliothèque `random`, avec la commande `sample(E, k)`, qui renvoie une liste de k éléments distincts choisis aléatoirement dans E.

```
from random import sample
def centres(E, k):
    return sample(E, k)  # k éléments distincts de E
```

2 On définit maintenant une fonction `partition` qui partitionne E autour des centres c_i pour $i = 0, ..., k-1$, représentés par une liste `centres`, selon la distance d. On assimile une classe à l'indice i du centre c_i, élément de la liste `centres = [c0,c1,...,ck-1]`. On peut pour cela utiliser la fonction

```
min(centres, key = lambda y:d(x,y))
```

que renvoie le centre c dont la distance $d(x, c)$ est minimale pour la donnée x. Cette ligne est équivalente au code :

```
c    = centres[0]
dist = d(x,c)
for y in centres[1:]:
    if d(x,y) < dist:
        c    = y
        dist = d(x,y)
return c
```

On peut ensuite coder la fonction `partition` de la façon suivante.

```
def partition(E, centres, d):
    """
    Entrée : ensemble d'apprentissage E (list), centres de classe centres (list),
             distance d (function)
    Action : partitionne E autour des centres selon la distance d
    Sortie : partition p (dict)
    """
    p = {}                      # initialisation partition = dictionnaire vide
    for x in E:                 # pour chaque donnée d'apprentissage x
        c    = min(centres, key = lambda y:d(x,y))  # centre le plus proche de x
        p[x] = centres.index(c)                     # classe attribuée à x
    return p
```

Une fois une partition définie, nous devons ré-évaluer les centres comme barycentre des classes E_i. On rappelle que le barycentre B d'un ensemble de points $\{M_1, ..., M_N\}$ est donné par

$$\overrightarrow{OB} = \frac{1}{n}\sum_{j=1}^{N}\overrightarrow{OM_j}. \tag{5}$$

où O est l'origine. Si les points appartiennent au plan et sont de coordonnées $M_j = (x_{1j}, x_{2j})$, on peut projeter sur les axes de sorte que $B = (b_1, b_2)$ avec

$$b_1 = \frac{1}{N}\sum_{j=1}^{N} x_{1j} \quad \text{et} \quad b_2 = \frac{1}{N}\sum_{j=1}^{n} x_{2j}. \tag{6}$$

On définit donc une fonction `bary` qui calcule le barycentre d'un ensemble de points.

```
def bary(points):
    """
    Entrée : points = liste de points de dimension 2
    Action : calcule le barycentre des points
    Sortie : b = barycentre des points
    """
    b = [0,0]                # initialisation du barycentre
    for x in points:         # boucle sur les points
        b[0] = b[0] + x[0]   # addition coordonnée x1
        b[1] = b[1] + x[1]   # addition coordonnée x2
    return b/len(points)     # barycentre = somme divisée par N
```

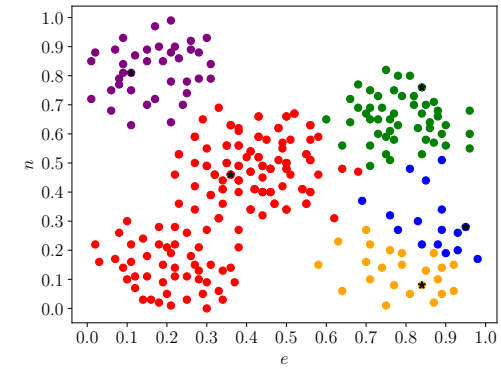
Exemple A2.9 : Classement non supervisé d'articles de presse

Des informaticiens travaillant pour un média connu souhaitent élaborer un algorithme de classement des articles de presse pour le site internet du magazine. Ils souhaitent répartir les articles en $k = 5$ thèmes, sans savoir comment les définir. On modélise un article de presse par un point du plan $y = (e, n)$ où $0 \leq e \leq 1$ représente la quantité d'équations dans le texte et $0 \leq n \leq 1$ est la quantité de noms propres. On dispose d'un ensemble d'apprentissage `articles` contenant les $N = 280$ articles de presse.

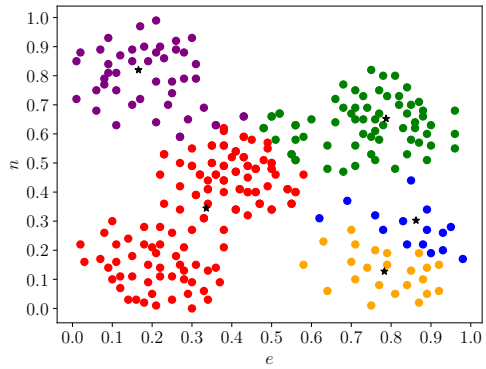
1) Implémenter la fonction `moy(E, k, d, ni)` qui applique l'algorithme des k-moyennes à un

ensemble d'apprentissage E avec une distance d et un nombre d'itérations n_i.

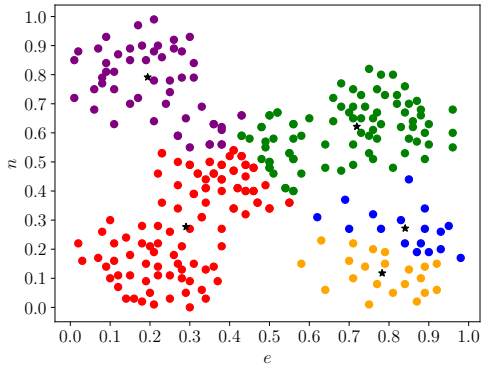
2) On suppose avoir défini la fonction `d` qui calcule la distance euclidienne. On effectue l'appel `moy(articles, 5, d, 10)` et on représente ci-dessous la partition des données à chaque itération. Commenter.



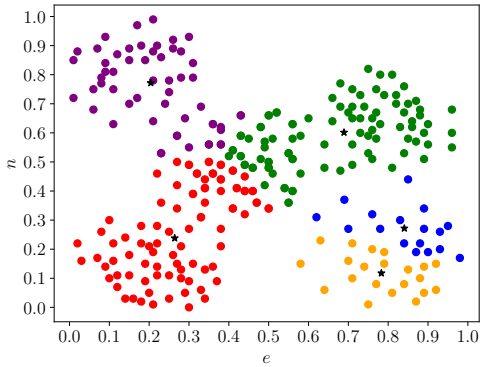
(a) $i = 0$



(b) $i = 1$

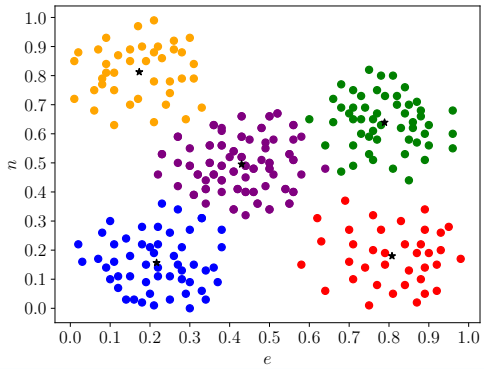


(c) $i = 7$

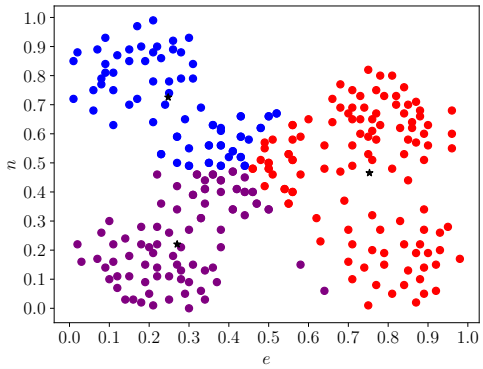


(d) $i = 10$

3) On représente ci-dessous la partition des données pour différentes situations : (a) un autre appel de `moy(articles, 5, d, 10)`, (b) un appel de `moy(articles, 3, d, 10)`. Commenter.



(a)



(b)

Solution

1)
```
def moy(E, k, d, ni):
    c = sample(E, k)                    # centres aléatoires
    for j in range(ni):                 # itération du partionnement
        p = partition(E, c, d)          # partionnement autour des centres
        for i in p.values():                    # boucle sur classe i
            classe = [x for x in p if p[x] == i] # ensemble correspondant
            c[i]   = bary(classe)               # ré-évaluation centres
    return p, c                         # retourne partition et centres
```

2) L'algorithme des k-moyennes dépend de plusieurs paramètres.

- *Le nombre de classes k. Si k est trop petit, des données sans similarité peuvent être classés dans une même catégorie. Si k est trop grand, les classes définies peuvent ne pas représenter des caractéristiques pertinentes des données.*
- *Le nombre d'itérations. Soit le nombre d'itérations est fixé à l'avance. Soit la boucle est illimité et s'arrête lorsque les centres ne sont plus modifiées ; l'algorithme a alors convergé vers une partition stable.*
- *L'algorithme dépend aussi fortement de l'initialisation des centres. Si les centres sont initialisés aléatoirement, la partition finale peut être différente d'une application de l'algorithme à l'autre sur un même jeu de données.*